



**ANGLO-CHINESE JUNIOR COLLEGE
JC2 PRELIMINARY EXAMINATION**

Higher 2

COMPUTING

9569/02

Paper 2 (Lab-based)

12 August 2026

3 hours

Additional Materials: Electronic version of ALL_WORDS.txt data file
 Electronic version of BOOKING.txt data file
 Electronic version of COACH.txt data file
 Electronic version of MEMBER.txt data file
 Insert Quick Reference Guide

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to **6** marks out of 100 will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [] at the end of each question or part question. The total number of marks for this paper is 100.

This document consists of **11** printed pages and **1** blank page.



Anglo-Chinese Junior College

[Turn Over

Instruction to candidates:

Your program code and output for each of Tasks 1, 2 and 3 should be saved in a single .ipynb file using Jupyter Notebook. For example, your program code and output for Task 1 should be saved as:

TASK1_<your name>_<centre number>_<index number>.ipynb

Make sure that each of your .ipynb files shows the required output in Jupyter Notebook.

Task 1

Name your Jupyter Notebook as:

TASK1_<your name>_<centre number>_<index number>.ipynb

Positive integers can be categorised according to properties of their factors. In particular, consider the sum of the factors of the integer which are less than the integer itself.

- If the sum of the factors is equal to the integer, we say that the integer is a perfect number.
 - o For example, 6 is perfect because $1 + 2 + 3 = 6$.
- If the sum of the factors is larger than the integer, we say that the integer is an abundant number.
 - o For example, 12 is abundant because $1 + 2 + 3 + 4 + 6 > 12$.
- If there are two distinct positive integers x and y such that the sum of the factors of x is equal to y , and the sum of the factors of y is equal to x , we say that x and y are amicable numbers.
 - o For example, the factors of 220 are 1, 2, 4, 5, 10, 11, 20, 22, 44, 55, and 110 which sum to 284. The factors of 284 are 1, 2, 4, 71 and 142 which sum to 220.
- If there is an integer x_1 such that the factors of x_1 sum to x_2 , and the factors of x_2 sum to x_3 , etc, until a term x_n is reached such that the factors of x_n sum to x_1 , we say that the numbers $x_1, x_2, \dots, x_n$ are sociable numbers.
 - o For example, $x_1 = 12496$, $x_2 = 14288$, $x_3 = 15472$, $x_4 = 14536$, $x_5 = 14264$ and the sum of factors of x_5 is x_1 . Therefore, these five numbers are sociable numbers.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

In [1]:

#Task 1.1
Program code

 Output:

Task 1.1

Write a function `factor_sum(x)` which returns the sum of factors of the integer x which are at least 1 and smaller than x . [3]

Task 1.2

Write program code to ask a user to input a positive integer k . The program should then print out the first k perfect numbers.

Test your code with $k = 4$. Print the output. [6]

Task 1.3

Write program code to find and print the smallest odd abundant number. [4]

Task 1.4

Write program code to ask a user to input a positive integer k . The program should then print out the smaller number of the first k pairs of amicable numbers. [5]

Test your code with $k = 4$. Print the output.

Task 1.5

Write a function `status(x)` which returns a string describing what type of positive integer x is.

Value of x	Expected output
28	"Perfect"
284	"Amicable"
12496	"Sociable"
10	"Nil"
562	"Nil"
608	"Nil"

Test your code on the cases above. Print your output. [7]

Save your Jupyter Notebook for Task 1.

Task 2

Name your Jupyter Notebook as:

TASK2_<your name>_<centre number>_<index number>.ipynb

In a single-player word guessing game, players attempt to guess a secret n-letter word. Assume all input strings consist of uppercase English letters, and the player always guesses an n-letter word.

After submitting a guess, the player receives an evaluation array of length n containing feedback indicators based on its comparison to the secret word.

- Each integer in the evaluation array is either 2, 1 or 0. The integer at a certain index in the evaluation array gives information about how close the guess is to the secret word.
- If a letter in the guess matches the letter in the secret word at the same index, the evaluation array would contain 2 at that index. These letters are no longer considered in the next step.
- For the remaining letters in the guess, if they match a letter at the secret word in a different index, the evaluation array would contain 1 at that index. These letters are no longer considered in the next step.
- Any remaining letters in the guess would correspond to a 0 in the evaluation array at that index.

For example, if the secret word is 'APPLE' and the guess is 'PUPPY', the evaluation array is [1, 0, 2, 0, 0].

- The "P" at index 2 in the guess matches the "P" at index 2 in the secret word, so the evaluation array contains 2 at index 2.
- The "P" at index 0 in the guess matches the "P" at index 1 in the secret word, so the evaluation array contains 1 at index 0.
- The "P" at index 3 in the guess does not have any remaining letters in the secret word to match with, so the the evaluation array contains 0 at index 3.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

In [1]: *#Task 2.1*
Program code
 Output:

Task 2.1

Write a function `right_index(secret, guess)` that takes in two strings, `secret` and `guess`, of equal length `n`.

The function must return a tuple containing two items:

- A partial evaluation array of length `n` with 2 at indices where the letters in `secret` and `guess` match, and `None` otherwise;
- A dictionary mapping each letter in `secret` to its frequency count, after subtracting the letters which have already been matched.

For example, `right_index("APPLE", "PUPPY")` should return

```
([None, None, 2, None, None], {"A": 1, "P": 1, "L": 1, "E": 1})
```

because there are two "P"s in "APPLE" and one of them has already been matched. [5]

Task 2.2

Write a function `wrong_index(guess, partial_eval, remaining_counts)` that takes the guess string, along with the partial evaluation array and the dictionary of remaining letter counts obtained from Task 2.1 as `partial_eval` and `remaining_counts`, respectively.

The function must update `partial_eval` by assigning 1 to the indices where the letters in `guess` match with a letter in `secret` at a different index, and also update the letter frequencies in `remaining_counts` by subtracting these matches. It should then return the updated evaluation array. [5]

Task 2.3

Write a function `score_word_guess(secret, guess)` that integrates your logic from Tasks 2.1 and 2.2 to construct the evaluation array.

The function must

- Call `right_index()` to obtain the partial evaluation array and letter counts after matching the letters in the correct index;
- Call `wrong_index()` to update the evaluation array and letter counts after matching the letters in different indices;
- Replace any remaining `None` elements in the evaluation array with 0;
- Return the final evaluation array. [3]

Task 2.4

Write a function `word_game(secret_word)` that integrates your logic from Task 2.1, 2.2 and 2.3 to construct the single-player word guessing game.

The game must:

- Run for 5 rounds or until the player guesses `secret_word` correctly, whichever comes first;
- Prompt the user for a guess during each round and display the resulting evaluation array;
- Display an appropriate message if the player guesses `secret_word` correctly (all elements in the evaluation array are 2);
- Display an appropriate message and reveal `secret_word` if the player runs out of attempts. [6]

Task 2.5

The file `ALL_WORDS.txt` contains all valid words to be used for the game, one on each line.

Write a function `filter_dictionary(guess, feedback)` that takes in a string `guess` and an evaluation array `feedback`, reads the file `ALL_WORDS.txt` and returns a list containing all possible secret words that are valid based on the input parameters `guess` and `feedback`. [5]

Save your Jupyter Notebook for Task 2.

Task 3

Name your Jupyter Notebook as:

TASK3_<your name>_<centre number>_<index number>.ipynb

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 3.1
        Program code
        Output:
```

An online assessment platform logs student exam submission. Each submission record contains:

- student_name : STRING
- score : INTEGER (0 to 100 inclusive)
- timestamp : INTEGER (representing submission time in 24-hour format, e.g., 1015 for 10:15 AM)

The platform uses object-oriented programming to store these records in a linked list where each record is stored in a Node object.

The class Node contains four attributes:

- student_name: the node's student name data, initialized as None.
- score: the node's student score data, initialized as None.
- timestamp: an integer denoting time in 24-hour format, initialized as None.
- next: a pointer to the next Node, initialized as None

The class Node has the following methods:

- a constructor `__init__()` that initialises the four attributes
- `set_student_name()` that assigns a value to `student_name`
- `set_score()` that assigns a value to `score`
- `set_next()` that assigns a value to `next`
- `set_timestamp()` that assigns a value to `timestamp`
- `get_student_name()` that returns `student_name`
- `get_score()` that returns `score`
- `get_next()` that returns `next`
- `get_timestamp()` that returns `timestamp`

Task 3.1

Write program code to declare the class Node, its constructor and its methods.

[4]

The class `LinkedList` contains one attribute:

- `head` reference to the first node, initialized as `None` for an empty linked list

The class `LinkedList` has the following methods:

- a constructor `__init__()` that initialises the attribute `head`
- `length()` that returns the number of nodes in the linked list
- `pop()` which removes and returns the first node from the linked list, if any
- `display()` which traverses `LinkedList` from `head` to the last node in `LinkedList` and displays the student's name, score, and timestamp for each student record
- `add_node()` takes in a `Node` object and adds it to the end of the linked list
- `insert(n, node)` that inserts `node` at index `n` (under 0-indexing)
 - If `n = 0`, then `node` is inserted into the linked list as its first node
 - If `n = k` where `k` is the current length of the linked list, then `node` is inserted into the linked list as its last node
 - The method does nothing if `n < 0` or `n > k`

Task 3.2

Write program code to declare the class `LinkedList`, its constructor and its methods. [10]

Task 3.3

Write program code to:

1. Instantiate a `LinkedList` object named `ExamRecord`.
2. Add the following nodes into `ExamRecord`:
 - `("Alice", 85, 1015)`
 - `("Bob", 91, 1002)`
 - `("Charlie", 86, 1001)`
 - `("David", 70, 1030)`
 - `("Eve", 92, 1008)`

Alice is to be placed as the first node in the linked list, followed by Bob, Charlie, David and finally Eve. [2]

Task 3.4

For this task, do **not** use Python's built-in sorting functions (e.g., `sort()` or `sorted()`).

The assessment platform requires submission records to be sorted by timestamp.

Write program code to:

1. Obtain the list of tuples containing the student's name, score, and timestamp for each student record from `ExamRecord` by calling its `display()` method.
2. Instantiate an empty `LinkedList` object named `SortedRecord`.
3. Create a new `Node` object for each tuple in Step 1.
4. Using an insertion sort algorithm, and utilizing `SortedRecord.insert(n, node)` or otherwise, add each node into `SortedRecord` so that the records are arranged in ascending order of timestamp.
5. Traverse `SortedRecord` and print the `student_name` of each node, one per line, in the sorted order. [7]

Save your Jupyter Notebook for Task 3.

Task 4

Name your Jupyter Notebook as:

TASK4_<your name>_<centre number>_<index number>.ipynb

A gym keeps records about its members and bookings. The gym wants to create a suitable database to store the data and allow them to run searches for specific data. The database will have three tables: a table to store data about the members, a table about the bookings, and a table about the coaches. The fields in each table are:

Members:

- MemberID – member's unique number, for example, 101
- Name – member's name

Coaches:

- CoachID – coach's unique number, for example, 2
- Name – coach's name

Booking:

- BookingID – the purchase's unique ID, for example, 1001
- BookingDateTime – the date and time of the booking in YYYYMMDDHHMM format
- Venue – the location of the gym branch for that booking
- MemberID – the MemberID of the gym member who made the booking
- CoachID – the CoachID of the coach if the member requested for one, and 0 otherwise

For each of the sub-tasks 4.1 and 4.2, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 4.1
        Program code
        Output:
```

Task 4.1

Write a Python program that uses SQL code to create the database GYM with the three tables given. Define the primary and foreign keys for each table. [6]

Task 4.2

The text files MEMBER.txt, COACH.txt and BOOKING.txt contains the comma-separated values for each of the tables in the database.

Write a Python program to

- read the data from each file and then store each item of data in the correct place in the database.
- insert an additional record into the Coaches table where the CoachID is 0 and the Name is NULL.

[5]

Task 4.3

Write a Python program and the necessary files to create a web application that:

- displays a HTML form asking for a member name and a date in YYYYMMDD format, then
- creates and returns a HTML document that enables the web browser to display a table tabulating the booking date and time, location and coach name (if any) of any bookings made by the member on or after the date entered in the form.

Save your program code as

TASK4_3_<your name>_<centre number>_ <index number>.py

with any additional files/subfolders as needed in a folder named

TASK4_3_<your name>_<centre number>_ <index number>

Run the web application. Save the output of the program when "Sam Seng" is entered as the member name and 20260312 is entered as the date.

Save the webpage output as:

TASK4_3_<your name>_<centre number>_ <index number>.html

[11]